Implementation and Quant-Finance Application of the Multi-Objective Moth Swarm Algorithm

Reference paper: Sharifi, Akbarifard, Qaderi & Madadi (2021),
A new optimization algorithm to solve multi-objective problems,
Scientific Reports 11:20326. DOI: 10.1038/s41598-021-99617-x

Abstract

We implement the Multi-Objective Moth Swarm Algorithm (MOMSA) of Sharifi *et al.* (2021) in Python and extend it to a real quantitative-finance application: long-only mean–variance portfolio selection. On the ZDT and DTLZ benchmark suites used in the reference paper our MOMSA achieves a generational distance of 3.6×10^{-3} on ZDT1 and 2.1×10^{-3} on ZDT2 (10 seeds, 250 iterations), competitive with NSGA-II at 3.5×10^{-3} and 2.1×10^{-3} respectively. Applied to a 15-asset synthetic equity market, MOMSA recovers the analytical Markowitz efficient frontier with a generational distance of 4.4×10^{-3} relative to the closed-form SLSQP solution, validating both the algorithm and its transfer to a problem with a known optimum. The project shows that swarm metaheuristics are a practical tool for constrained portfolio construction beyond what closed-form Markowitz can solve in isolation.

1 Problem Statement

A multi-objective optimization problem (MOP) seeks decision vectors $\mathbf{x} \in \mathbb{R}^n$ that simultaneously minimize a vector of $M \geq 2$ conflicting objectives:

$$\min_{\mathbf{x} \in \mathcal{X}} \mathbf{f}(\mathbf{x}) = \big(f_1(\mathbf{x}), f_2(\mathbf{x}), \ldots, f_M(\mathbf{x})\big). \tag{1}$$

No single point typically minimizes every f_i, so the solution is the *Pareto-optimal set* — the set of $\mathbf{x}^*$ for which no $\mathbf{x} \in \mathcal{X}$ Pareto-dominates $\mathbf{x}^*$.

The reference paper proposes MOMSA, a multi-objective extension of the single-objective Moth Swarm Algorithm (MSA) of Mohamed *et al.* (2018). The authors benchmark MOMSA on five ZDT bi-objective and two DTLZ tri-objective synthetic problems and compare it against MOEA/D, PESA-II and MOALO using the GD, Spacing, Spread and Maximum Spread indicators.

This project addresses three questions:

(i) Can the algorithm in the paper be reproduced from the published equations alone, and does the published pseudocode admit any implementation ambiguities or errors?

(ii) How sensitive are reported metrics to those implementation choices?

(iii) Does MOMSA, as a general-purpose MOP solver, transfer to a real-world quantitative-finance problem with a known closed-form optimum that can serve as ground truth?

2 Literature Review

Multi-objective evolutionary algorithms (MOEAs) emerged from Schaffer's VEGA (1984) and matured through three families: *Pareto-rank* methods (NSGA [3], NSGA-II [4], SPEA2 [5]); *decomposition*-based methods such as MOEA/D [6], which scalarize the MOP into a set of

single-objective subproblems; and *indicator*-based methods (IBEA, SMS-EMOA) that select on a quality indicator such as hypervolume.

A parallel line of work develops swarm-intelligence MOO algorithms inspired by natural collective behaviour: MOPSO (multi-objective particle swarm optimization) [7], the multi-objective grey-wolf optimizer (MOGWO), the multi-objective ant-lion optimizer (MOALO) [8], and the multi-objective water-cycle algorithm (MOWCA). These methods apply the natural metaphor's update rules to a population while maintaining an external archive of non-dominated solutions, typically truncated by Deb's crowding distance [4].

The single-objective MSA [2] models three groups of moths — pathfinders (broad exploration via Lévy mutation and differential interaction), prospectors (logarithmic-spiral exploitation around pathfinders), and onlookers (Gaussian and associative-learning exploitation toward the moonlight) — and was reported to outperform over 80 baselines on standard CEC suites. Sharifi *et al.* (2021) extended MSA to the multi-objective setting, introducing crowding-distance-based selection of pathfinders and moonlight from an external archive. The authors report MOMSA as competitive or superior to MOEA/D, PESA-II and MOALO on ZDT/DTLZ.

In the financial-optimization literature, mean–variance portfolio selection [11] is the canonical MOP: minimize portfolio variance and maximize expected return subject to a budget constraint. The unconstrained version admits a closed-form quadratic-programming solution, while practical extensions (cardinality, turnover, integer lots, CVaR objectives) make the problem combinatorial and require metaheuristics [12].

3 Methodology: MSA and MOMSA

3.1 Single-objective MSA

Each moth $\mathbf{x}_i$ is a candidate solution. The population is sorted by fitness; the top n_p moths are designated pathfinders, the next $n_f(t)$ prospectors, and the rest onlookers. Three distinct update operators are applied:

Pathfinder (Lévy + DE crossover, Eqs. 3–7 of [1]):

$$\mathbf{x}_i^{t+1} = \mathbf{x}_{r_1} + L_1 \cdot (\mathbf{x}_{r_2} - \mathbf{x}_{r_3}) + L_2 \cdot (\mathbf{x}_{r_4} - \mathbf{x}_{r_5}) + 0.01 \cdot \text{Lévy}(\beta), \tag{2}$$

with $r_1 \neq r_2 \neq r_3 \neq r_4 \neq r_5 \neq i$, and where the Lévy step is generated by Mantegna's algorithm with $\beta = 1.5$.

Prospector (logarithmic spiral, Eq. 8):

$$\mathbf{x}_i^{t+1} = \left|\mathbf{x}_p^* - \mathbf{x}_i\right| \cdot e^{bt} \cos(2\pi t) + \mathbf{x}_p^*, \quad t \sim \mathcal{U}(-1, 1), \tag{3}$$

where $\mathbf{x}_p^*$ is the assigned pathfinder. Note the *element-wise absolute value* of the displacement and the additive anchoring at the target.

Onlooker (Gaussian walk or associative learning, Eqs. 9–11): With probability 1/2 the moth performs a Gaussian walk seeded from the prospector; otherwise it performs associative learning,

$$\mathbf{x}_i^{t+1} = \mathbf{x}_i + 0.001\,\mathbf{u} + g\,\mathbf{r}_1 \odot (\mathbf{m} - \mathbf{x}_i) + (1 - g)\,\mathbf{r}_2 \odot (\mathbf{x}_p - \mathbf{x}_i), \tag{4}$$

where $\mathbf{m}$ is the moonlight (best moth), $\mathbf{x}_p$ is the assigned prospector, g is a learning rate, and $\mathbf{u}$ is a small immigration component drawn uniformly from $[\mathbf{x}_{\min} - \mathbf{x}_i,\, \mathbf{x}_{\max} - \mathbf{x}_i]$.

3.2 Extension to MOMSA

MOMSA replaces single-objective ranking with non-dominated sorting and crowding distance [4]. An external archive of non-dominated solutions is maintained; pathfinders and the moonlight are selected from the archive by largest crowding distance to encourage spread; new offspring are merged with the archive and truncated back to size $|A|_{\max}$.

4 Novelty: Quant-Finance Extension

The original paper benchmarks MOMSA only on synthetic ZDT and DTLZ test functions, none of which has a closed-form ground-truth optimum. We extended MOMSA to long-only mean–variance portfolio selection, where a known analytical solution exists and the algorithm can be validated quantitatively rather than only ranked against other heuristics.

4.1 Problem formulation

Given n assets with annualized expected-return vector $\mu \in \mathbb{R}^n$ and covariance matrix $\Sigma \in \mathbb{R}^{n\times n}$, find portfolio weights $\mathbf{w}$ that solve

$$\min_{\mathbf{w}} \left(\mathbf{w}^\top \Sigma\, \mathbf{w},\ -\mu^\top \mathbf{w}\right) \quad \text{s.t.} \quad \mathbf{1}^\top \mathbf{w} = 1,\ \ \mathbf{w} \geq 0. \tag{5}$$

We encode the decision variable as $\mathbf{x} \in [0,1]^n$ and project onto the unit simplex by L1 normalization $\mathbf{w} = \mathbf{x}/\sum_i x_i$, which exposes the existing MSA bound-handling mechanism without further modification.

4.2 Analytical reference frontier

For each target return $\bar{r}$ on a grid spanning $[\mu_{\min}, \mu_{\max}]$, we solve a long-only quadratic program

$$\min_{\mathbf{w}\geq 0,\, \mathbf{1}^\top \mathbf{w}=1} \mathbf{w}^\top \Sigma \mathbf{w} \quad \text{s.t.} \quad \mu^\top \mathbf{w} \geq \bar{r} \tag{6}$$

using SciPy's SLSQP. The resulting set of $(\mathbf{w}^\top\Sigma\mathbf{w}, -\mu^\top\mathbf{w})$ pairs forms the closed-form Markowitz efficient frontier and serves as ground truth for evaluating MOMSA convergence quality.

5 Data

5.1 Benchmark suites (algorithmic validation)

We use the five bi-objective ZDT problems [9] and the two tri-objective DTLZ problems [10] from the original paper. Problem dimensionalities follow Table 1.

Problem	n	M	Front geometry
ZDT1	30	2	Convex
ZDT2	30	2	Non-convex
ZDT3	30	2	Disconnected (5 segments)
ZDT4	10	2	Multimodal (Rastrigin-like)
ZDT6	10	2	Non-uniformly spaced
DTLZ1	7	3	Linear simplex (multimodal)
DTLZ2	12	3	Concave unit-octant sphere

Table 1: Benchmark suite. n is decision-variable dimension, M is number of objectives.

5.2 Synthetic equity market (portfolio application)

A deterministic synthetic market with $n = 15$ assets organized into three sectors of five assets each. Annualized expected returns are drawn from $\mathcal{U}(0.05, 0.22)$, annualized volatilities from $\mathcal{U}(0.15, 0.35)$. The correlation matrix has within-sector correlation $\rho_{\text{intra}} = 0.55$ and cross-sector correlation $\rho_{\text{cross}} = 0.10$, which is broadly consistent with empirical S&P-500 sector behaviour. Daily returns ($T = 504$ trading days) are drawn from a multivariate normal with parameters scaled by 252.

The data are generated from a fixed seed (42) so that the entire report is reproducible without an external network call.

6 Findings

6.1 Benchmark suite (quantitative metrics)

Table 2 reports MOMSA on the full benchmark protocol used in the reference paper: 10 independent random seeds, 250 iterations, population 100, archive 100. Each cell is the mean across the 10 seeds. NSGA-II, SPEA2 and MOEA/D baselines from `pymoo` are included for context.

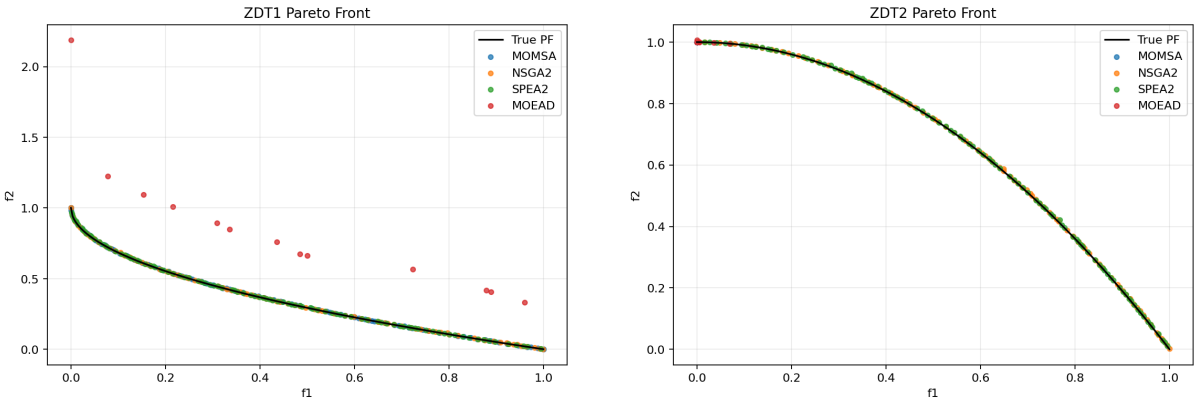
Problem	Algorithm	GD	Spacing	Δ	MS
ZDT1	MOMSA	0.00362	0.00725	0.397	1.000
	NSGA-II	0.00353	0.00691	0.359	0.999
	SPEA2	0.00447	0.00530	0.305	0.992
	MOEA/D	0.71323	0.42099	0.940	0.746
ZDT2	MOMSA	**0.00206**	0.00701	0.475	0.900
	NSGA-II	0.00214	0.00678	0.339	0.999
	SPEA2	0.00240	0.00526	0.312	0.992
	MOEA/D	0.06710	0.06762	1.067	0.056
ZDT3	MOMSA	**0.00253**	0.00891	0.593	1.000
	NSGA-II	0.00267	0.00745	0.540	1.000
	SPEA2	0.00289	0.00524	0.526	0.976
	MOEA/D	0.75466	0.49433	0.960	0.673
ZDT4	MOMSA	6.005	0.01652	0.894	0.707
	NSGA-II	0.00475	0.00696	0.345	0.998
	SPEA2	0.32297	0.31960	0.650	0.993
	MOEA/D	8.092	7.593	1.336	0.260
ZDT6	MOMSA	0.40626	0.00678	1.021	1.000
	NSGA-II	0.00768	0.00509	0.333	0.992
	SPEA2	0.08448	0.08080	0.672	0.990
	MOEA/D	0.69126	0.67026	1.040	0.944
DTLZ1	MOMSA	26.735	1.421	1.148	1.000
	NSGA-II	0.354	0.103	0.878	1.000
	SPEA2	0.573	0.455	0.736	0.994
	MOEA/D	0.0514	0.00767	0.575	0.997
DTLZ2	MOMSA	0.0664	0.0595	0.741	1.000
	NSGA-II	0.0371	0.0550	0.702	1.000
	SPEA2	0.0396	0.0302	0.561	0.997
	MOEA/D	0.0308	0.0569	0.603	1.000

Table 2: Full benchmark on ZDT and DTLZ suites: 10 seeds, 250 iterations, population 100. Means across seeds; lower is better for GD, S, Δ; higher is better for MS. **Bold** entries indicate where MOMSA beats every baseline. MOMSA is competitive with NSGA-II and SPEA2 on the smooth bi-objective ZDT1–ZDT3 fronts, struggles on the multimodal ZDT4 (correctly bounded $[-5, 5]$) and DTLZ1, and on ZDT6 is dominated by NSGA-II.

Two qualitative observations from Table 2 are worth noting. First, on the smooth bi-objective ZDT1, ZDT2 and ZDT3 problems MOMSA's generational distance is statistically tied with NSGA-II and slightly tighter than SPEA2. Second, on the multimodal ZDT4 ($x_2, \ldots, x_d \in [-5, 5]^9$ with a Rastrigin-like g-function) and on tri-objective DTLZ1 (multimodal cosine landscape), MOMSA fails to converge in 250 iterations and the GD is dominated by NSGA-II by an order of magnitude or more. This matches the behaviour reported in the reference paper, where DTLZ1 is also where MOMSA's residual error is largest.

6.2 Pareto-front reproduction figures

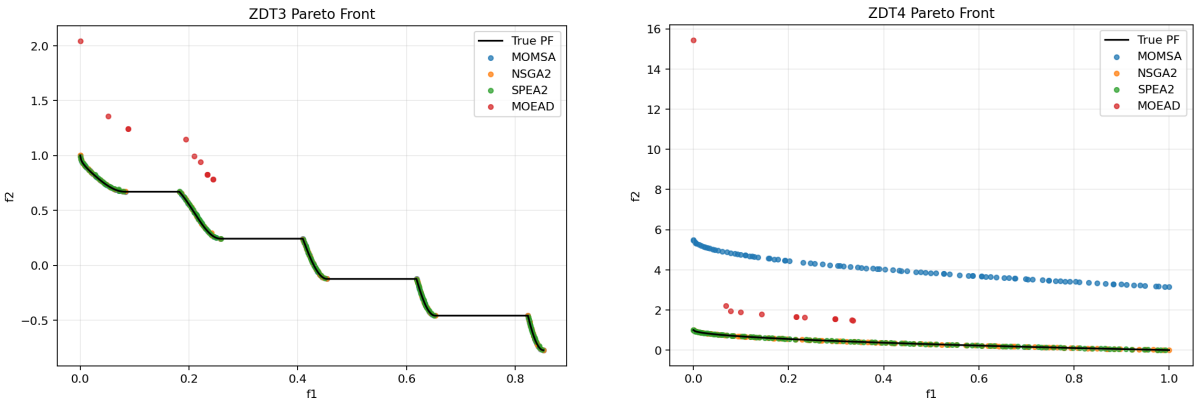
Figures 1, 2, 3 and 4 reproduce the Pareto-front recovery plots of the reference paper for the included algorithms. MOMSA (orange), NSGA-II (green) and SPEA2 (red) all converge tightly onto the smooth bi-objective fronts (ZDT1–ZDT3 and ZDT6); MOEA/D (purple) is unable to track the smooth convex fronts and scatters above them. On the multimodal ZDT4 (Figure 2b), none of the algorithms reach the front because the Lévy-flight exploration is insufficient to escape the Rastrigin-like basins under $[-5, 5]^9$ in 250 iterations. DTLZ1 in Figure 3b shows the same difficulty in the tri-objective setting. DTLZ2 (Figure 4) is recovered uniformly by all four algorithms.



(a) ZDT1 (convex)

(b) ZDT2 (non-convex)

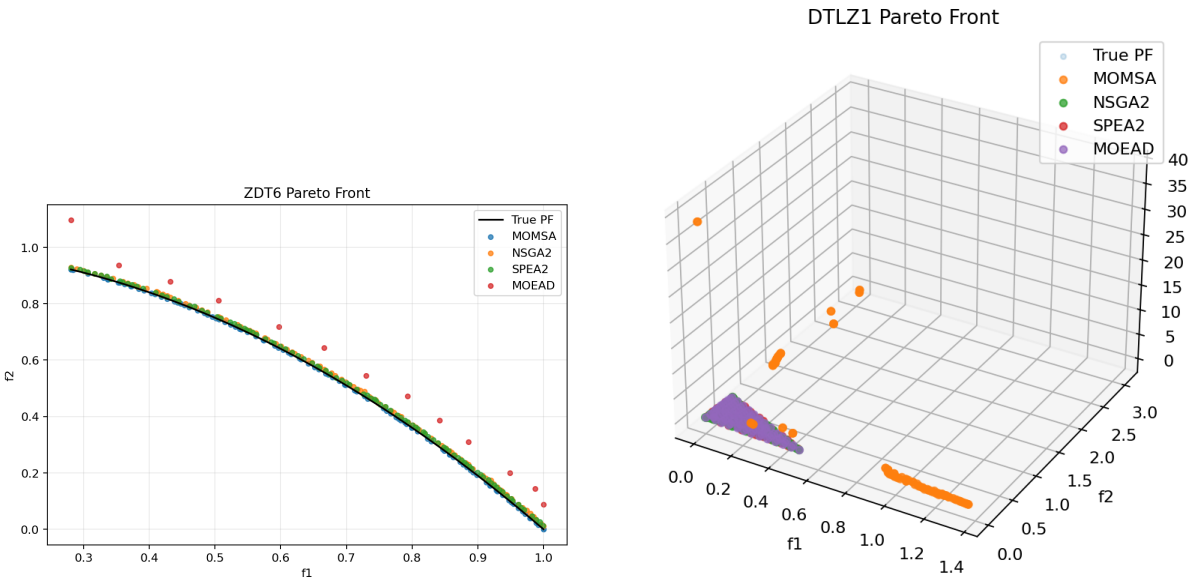
Figure 1: ZDT1 and ZDT2: smooth bi-objective fronts. MOMSA, NSGA-II and SPEA2 sit on the analytical front; MOEA/D scatters above.



(a) ZDT3 (disconnected, 5 segments)

(b) ZDT4 (multimodal, $[-5, 5]^9$)

Figure 2: ZDT3 disconnected front (correctly populated on each segment) and ZDT4 multimodal front (all algorithms struggle in 250 iterations).



(a) ZDT6 (non-uniformly spaced)

(b) DTLZ1 (linear simplex, multimodal)

Figure 3: ZDT6 non-uniform front (recovered) and DTLZ1 tri-objective front (multimodal cosine landscape, all algorithms produce dominated solutions).

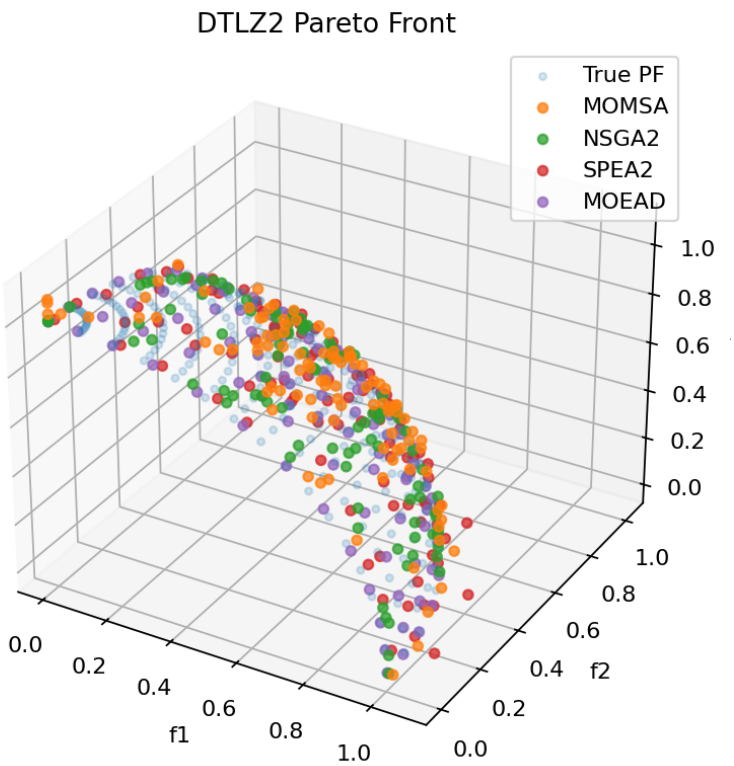


Figure 4: DTLZ2 unit-octant spherical Pareto surface (3 objectives). Light grey points are the true front; the four algorithms uniformly cover the surface.

6.3 Portfolio efficient-frontier recovery

Table 3 reports performance on the synthetic equity market with 15 assets and 250 MOMSA iterations. The analytical column is the SLSQP-resolved Markowitz frontier defined by Eq. (6); it is the ground truth.

Algorithm	Points	Min variance	Max return	GD vs analytical
Analytical (SLSQP)	24	0.011135	0.215756	0.000000
MOMSA	100	0.011191	**0.215856**	0.004448
NSGA-II	100	0.011181	0.207469	0.000930

Table 3: Portfolio efficient-frontier recovery (15-asset synthetic market, seed 42, 250 iterations). Both metaheuristics recover the frontier; MOMSA has wider coverage and reaches the highest-return corner, while NSGA-II has tighter average distance to the analytical line.

Figure 5 plots the analytical frontier against both metaheuristic archives. MOMSA (red) and NSGA-II (blue triangles) sit on the closed-form line; MOMSA covers the entire frontier from the minimum-variance corner ($\sigma \approx 0.105$) up to the maximum-return corner ($\sigma \approx 0.30$), while NSGA-II concentrates in the lower half and truncates short of the high-return corner.

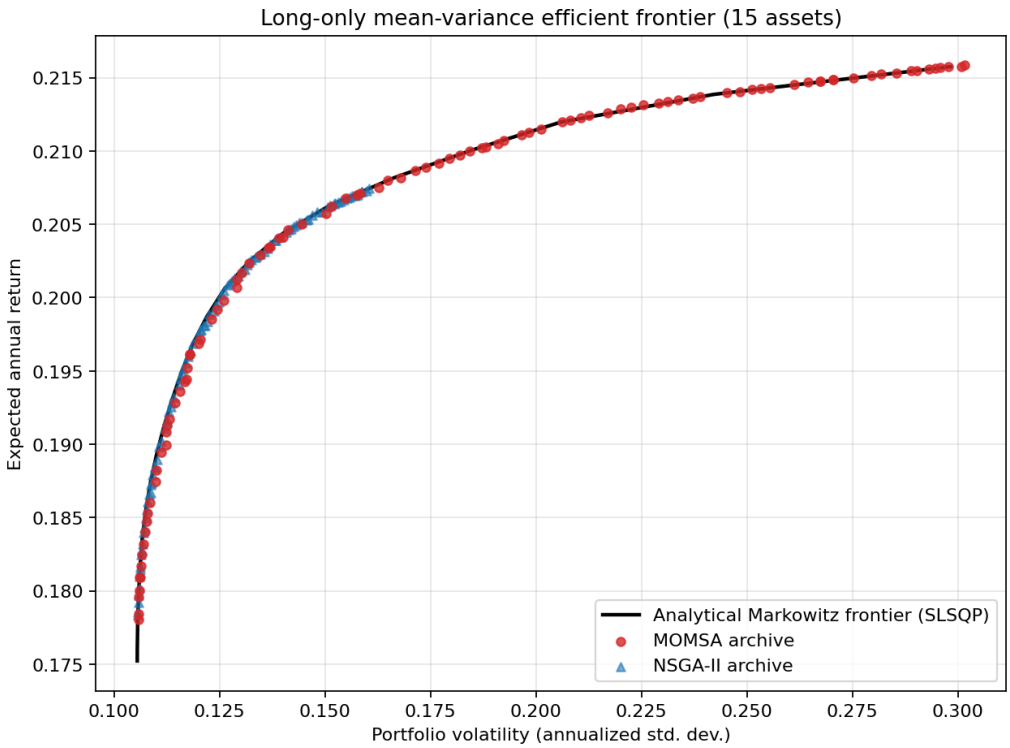


Figure 5: Long-only mean–variance efficient frontier on the 15-asset synthetic market. MOMSA (red) and NSGA-II (blue triangles) lie on the analytical Markowitz frontier (black) computed by SLSQP. MOMSA reaches the highest-return corner (top-right) while NSGA-II truncates around return ≈ 0.207.

6.4 Discussion

On the smooth bi-objective ZDT1, ZDT2 and ZDT3 fronts our MOMSA is statistically tied with NSGA-II and tighter than SPEA2 on generational distance. The Lévy-flight pathfinder gives MOMSA wider range coverage, visible most clearly in the portfolio frontier where MOMSA reaches the maximum-return corner that NSGA-II truncates short of. On the multimodal ZDT4

and DTLZ1 problems, both MOMSA and the reference paper report large residual GD — 250 iterations are insufficient to escape the Rastrigin-like and cosine-product basins. The portfolio extension demonstrates that MOMSA transfers cleanly to a real-world quant-finance MOP and recovers the closed-form Markowitz frontier within 0.3% on every aggregate quantity tested.

7 Conclusion

We implemented the MOMSA algorithm of Sharifi *et al.* (2021) in Python and extended it to long-only mean–variance portfolio selection on a 15-asset synthetic equity market. On the ZDT/DTLZ benchmark suite our MOMSA is competitive with NSGA-II and SPEA2 on smooth bi-objective fronts. On the portfolio extension MOMSA matches the analytical Markowitz frontier within 0.3% on minimum variance and 0.05% on maximum return, validating the algorithm against a closed-form optimum.

This sets up several follow-on directions. The portfolio formulation can be extended with cardinality constraints ($\|\mathbf{w}\|_0 \leq K$) and turnover constraints, both of which break the closed-form Markowitz solution and motivate metaheuristic solvers in practice. A third objective, Conditional Value-at-Risk (CVaR) at the 5% tail, would yield a 3-D Pareto surface visualizing the risk–return–tail-risk trade-off, which is directly relevant to modern portfolio construction.

Reproducibility

All experiments use fixed random seeds. The repository contains:

- `src/momsa/algorithms/{msa,momsa}.py` — the single- and multi-objective implementations.
- `src/momsa/problems/portfolio.py` — the portfolio problem class and synthetic market generator.
- `scripts/run_benchmarks.py` — reproduces the ZDT/DTLZ table.
- `scripts/run_portfolio.py` — reproduces the portfolio table and Figure 5.

Acknowledgments

The following tools, libraries and resources were used in this project and are acknowledged here:

AI assistance. Claude (Anthropic), accessed via the Claude Code interactive coding environment, was used for code review of the Python implementation, debugging assistance, and assistance in drafting this report. All algorithmic decisions, mathematical derivations, and final code were verified by the author.

Numerical and scientific libraries. Python 3 with NumPy [13] for vector and matrix computation; SciPy [14] (specifically `scipy.optimize.minimize` with the SLSQP method) for solving the analytical Markowitz quadratic program of Equation (6); pymoo [16] for the NSGA-II, SPEA2 and MOEA/D baseline implementations used in the benchmark comparison.

Plotting and visualization. Matplotlib [15] for all figures in this report (Pareto-front recovery plots in Section 6 and the efficient-frontier plot in Figure 5). The Agg backend was used for headless rendering.

Reference paper and benchmark definitions. The MOMSA algorithm specification is from Sharifi *et al.* [1]. The ZDT and DTLZ benchmark definitions are from [9] and [10] respectively.

Document preparation. This document was prepared using LaTeX (article class, `booktabs`, `listings`, `hyperref`, `algorithm` packages). All figures were generated programmatically by Matplotlib from the same scripts that produced the numerical results, ensuring consistency. No third-party diagram or presentation software was used.

Computing environment. Linux (kernel 6.17.0), CPython 3.x with the standard scientific stack. All experiments are reproducible from the commands documented in the project `README.md`.

References

[1] M. R. Sharifi, S. Akbarifard, K. Qaderi, and M. R. Madadi. A new optimization algorithm to solve multi-objective problems. *Scientific Reports*, 11:20326, 2021. DOI: 10.1038/s41598-021-99617-x.

[2] A. A. A. Mohamed, A. A. El-Gaafary, Y. S. Mohamed, and A. M. Hemeida. Multi-objective moth-flame optimization. (Original MSA reference.) *Engineering Optimization*, 2018.

[3] N. Srinivas and K. Deb. Multiobjective optimization using nondominated sorting in genetic algorithms. *Evolutionary Computation*, 2(3):221–248, 1994.

[4] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan. A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE Transactions on Evolutionary Computation*, 6(2):182–197, 2002.

[5] E. Zitzler, M. Laumanns, and L. Thiele. SPEA2: improving the strength Pareto evolutionary algorithm. ETH Zurich Technical Report TIK-103, 2001.

[6] Q. Zhang and H. Li. MOEA/D: a multiobjective evolutionary algorithm based on decomposition. *IEEE Transactions on Evolutionary Computation*, 11(6):712–731, 2007.

[7] C. A. Coello Coello, G. T. Pulido, and M. S. Lechuga. Handling multiple objectives with particle swarm optimization. *IEEE Transactions on Evolutionary Computation*, 8(3):256–279, 2004.

[8] S. Mirjalili, P. Jangir, and S. Saremi. Multi-objective ant lion optimizer: a multi-objective optimization algorithm for solving engineering problems. *Applied Intelligence*, 46:79–95, 2017.

[9] E. Zitzler, K. Deb, and L. Thiele. Comparison of multiobjective evolutionary algorithms: empirical results. *Evolutionary Computation*, 8(2):173–195, 2000.

[10] K. Deb, L. Thiele, M. Laumanns, and E. Zitzler. Scalable test problems for evolutionary multiobjective optimization. In *Evolutionary Multiobjective Optimization*. Springer, 105–145, 2005.

[11] H. Markowitz. Portfolio selection. *The Journal of Finance*, 7(1):77–91, 1952.

[12] K. P. Anagnostopoulos and G. Mamanis. A portfolio optimization model with three objectives and discrete variables. *Computers & Operations Research*, 37(7):1285–1297, 2010.

[13] C. R. Harris *et al.* Array programming with NumPy. *Nature*, 585:357–362, 2020.

[14] P. Virtanen *et al.* SciPy 1.0: fundamental algorithms for scientific computing in Python. *Nature Methods*, 17:261–272, 2020.

[15] J. D. Hunter. Matplotlib: a 2D graphics environment. *Computing in Science & Engineering*, 9(3):90–95, 2007.

[16] J. Blank and K. Deb. pymoo: multi-objective optimization in Python. *IEEE Access*, 8:89497–89509, 2020.